

Week 2 - Class 0 - SLE mini project

SLE mini project (continued)

Repeating some of the previous class (week 1)

If necessary use the `BiocManager::install("GEOquery")` function call to install the package. We will also use **dplyr** and **ggplot2** packages to help explore this dataset further.

NOTE: **select** function confuses R sometimes. When you refer to the **select** within **dplyr** it is a good idea to use **dplyr::select** instead of **select**

```
# chunk 1: read, clean, filter the data for a specific cell type

# Load the required packages
library(GEOquery)
```

Loading required package: Biobase

Loading required package: BiocGenerics

Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

anyDuplicated, aperm, append, as.data.frame, basename, cbind,
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,

```
match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,  
Position, rank, rbind, Reduce, rownames, sapply, setdiff, sort,  
table, tapply, union, unique, unsplit, which.max, which.min
```

Welcome to Bioconductor

```
Vignettes contain introductory material; view with  
'browseVignettes()'. To cite Bioconductor, see  
'citation("Biobase")', and for packages 'citation("pkgname")'.
```

```
Setting options('download.file.method.GEOquery'='auto')
```

```
Setting options('GEOquery.inmemory.gpl'=FALSE)
```

```
library(dplyr)
```

Attaching package: 'dplyr'

The following object is masked from 'package:Biobase':

```
combine
```

The following objects are masked from 'package:BiocGenerics':

```
combine, intersect, setdiff, union
```

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

```
library(ggplot2)  
library(DESeq2)
```

Loading required package: S4Vectors

Loading required package: stats4

Attaching package: 'S4Vectors'

The following objects are masked from 'package:dplyr':

first, rename

The following objects are masked from 'package:base':

expand.grid, I, unname

Loading required package: IRanges

Attaching package: 'IRanges'

The following objects are masked from 'package:dplyr':

collapse, desc, slice

The following object is masked from 'package:grDevices':

windows

Loading required package: GenomicRanges

Loading required package: GenomeInfoDb

Loading required package: SummarizedExperiment

Loading required package: MatrixGenerics

Loading required package: matrixStats

Attaching package: 'matrixStats'

The following object is masked from 'package:dplyr':

count

The following objects are masked from 'package:Biobase':

anyMissing, rowMedians

Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

colAlls, colAnyNAs, colAnys, colAvgPerRowSet, colCollapse,
colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
colWeightedMeans, colWeightedMedians, colWeightedSds,
colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgPerColSet,
rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
rowWeightedSds, rowWeightedVars

The following object is masked from 'package:Biobase':

rowMedians

```
path_data <- file.path("../", "data")
path_output <- file.path("../", "output")
dir.create(path_output, showWarnings = FALSE)
```

```
# Complete the code with the GEO ID
gse <- getGEO("GSE149050")
```

Found 1 file(s)

GSE149050_series_matrix.txt.gz

```
gse
```

```
$GSE149050_series_matrix.txt.gz
ExpressionSet (storageMode: lockedEnvironment)
assayData: 0 features, 288 samples
  element names: exprs
protocolData: none
phenoData
  sampleNames: GSM4489145 GSM4489146 ... GSM4489432 (288 total)
  varLabels: title geo_accession ... visit number:ch1 (64 total)
  varMetadata: labelDescription
featureData: none
experimentData: use 'experimentData(object)'
  pubMedIds: 33674349
Annotation: GPL16791
```

```
# Our gse list has only one entry
metadata <- pData(phenoData(gse[[1]]))
dim(metadata)
```

```
[1] 288 64
```

```
grep("characteristics", colnames(metadata), value=TRUE)
```

```
[1] "characteristics_ch1"      "characteristics_ch1.1"  "characteristics_ch1.2"
[4] "characteristics_ch1.3"    "characteristics_ch1.4"  "characteristics_ch1.5"
[7] "characteristics_ch1.6"    "characteristics_ch1.7"  "characteristics_ch1.8"
[10] "characteristics_ch1.9"    "characteristics_ch1.10"
```

```
table(metadata$characteristics_ch1)[1]
```

disease state: healthy control

85

```
table(metadata$characteristics_ch1.2, metadata$characteristics_ch1.7)
```

	gender: Female	gender: Male
cell type: B cells	31	1
cell type: cDC	31	1
cell type: cMo	117	2
cell type: pDC	32	1
cell type: PMN	35	1
cell type: T cells	35	1

```
#metadata.cDC <- filter(metadata, characteristics_ch1.2 == "cell type: cDC" &
#                           characteristics_ch1.6 == "visit number: 1")
#head(metadata.cDC[,1:3])
```

```
metadata.cmo <- filter(metadata, characteristics_ch1.2 == "cell type: cMo" &
head(metadata.cmo[,1:3])
```

	title	geo_accession	status
GSM4489314	170_S3004a_IFNpos_cMo	GSM4489314	Public on Feb 01 2021
GSM4489318	174_S1055a_IFNpos_cMo	GSM4489318	Public on Feb 01 2021
GSM4489321	177_S1082a_IFNpos_cMo	GSM4489321	Public on Feb 01 2021
GSM4489322	178_S3008a_IFNpos_cMo	GSM4489322	Public on Feb 01 2021
GSM4489323	179_S3011a_IFNpos_cMo	GSM4489323	Public on Feb 01 2021
GSM4489325	181_S1065a_IFNpos_cMo	GSM4489325	Public on Feb 01 2021

```
# TO DO: Build up this example by piping to head() after each line
metadata.subset <- metadata.cmo %>%
  dplyr::select(title,
    disease_state = characteristics_ch1,
    ifn_status = characteristics_ch1.3,
    patient_id = characteristics_ch1.4) %>%
  mutate(disease_state = gsub("disease state: ", "", disease_state)) %>%
```

```
mutate(ifn_status = gsub("ifn status: ", "", ifn_status)) %>%
mutate(patient_id = gsub("patientuid: ", "", patient_id)) %>%
mutate(state = recode(disease_state,
                      "healthy control" = "control",
                      "systemic lupus erythematosus (SLE)" = "lupus") )

table(metadata.subset$state)
```

```
control  lupus
      24    64
```

```
counts.all <- read.delim(file.path(path_data, "GSE149050_Bulk_Human_RawCounts.txt"), check
head(counts.all[,1:3])
```

```
      001_L0038_HC_T 002_L0088fresh_HC_T 003_L0140_HC_T
5S_rRNA           2                   0             2
5_8S_rRNA         0                   0             0
7SK               1                   1             2
A1BG              53                  56            105
A1BG-AS1          7                   24             46
A1CF              0                   2              2
```

```
# below is one of the data objects that will go into DESeq
counts.subset <- counts.all %>% dplyr::select(metadata.subset$title)
head(counts.subset[,1:4])
```

```
      170_S3004a_IFNpos_cMo 174_S1055a_IFNpos_cMo 177_S1082a_IFNpos_cMo
5S_rRNA                   11                     5                 3
5_8S_rRNA                 0                     0                 0
7SK                       0                     3                 0
A1BG                      47                    34                39
A1BG-AS1                  29                    15                 2
A1CF                      2                     0                 0

      178_S3008a_IFNpos_cMo
5S_rRNA                   14
5_8S_rRNA                 0
7SK                       0
```

A1BG	118
A1BG-AS1	51
A1CF	0

DESeq

For DESeq we need to do some book-keeping and get the data in the correct format. First double check on column correspondences between counts and metadata

```
# chunk 2: sanity check
all(colnames(counts.subset) == metadata.subset$title)
```

```
[1] TRUE
```

```
all(c(1,2,3,4) == c(1,2,3,4))
```

```
[1] TRUE
```

Finally, DESeq needs our sample names as rownames of the metadata.

```
# chunk 3: shuffle columns around so DESeq is happy
# Remove the title column then use it as rownames
colData <- metadata.subset[,-1] # dropping a column
rownames(colData) <- metadata.subset[,1] # keeping only that column
head(colData)
```

	disease_state	ifn_status	patient_id
170_S3004a_IFNpos_cMo	systemic lupus erythematosus (SLE)	IFNpos	S3004
174_S1055a_IFNpos_cMo	systemic lupus erythematosus (SLE)	IFNpos	S1055
177_S1082a_IFNpos_cMo	systemic lupus erythematosus (SLE)	IFNpos	S1082
178_S3008a_IFNpos_cMo	systemic lupus erythematosus (SLE)	IFNpos	S3008
179_S3011a_IFNpos_cMo	systemic lupus erythematosus (SLE)	IFNpos	S3011
181_S1065a_IFNpos_cMo	systemic lupus erythematosus (SLE)	IFNpos	S1065
	state		
170_S3004a_IFNpos_cMo	lupus		
174_S1055a_IFNpos_cMo	lupus		
177_S1082a_IFNpos_cMo	lupus		
178_S3008a_IFNpos_cMo	lupus		
179_S3011a_IFNpos_cMo	lupus		
181_S1065a_IFNpos_cMo	lupus		

Now we can setup our DESeq object with the long but appropriately named `DESeqDataSetFromMatrix()` function:

```
# chunk 3: shuffle columns around so DESeq is happy

dds <- DESeqDataSetFromMatrix(countData = counts.subset,
                              colData = colData,
                              design = ~state)
```

Warning in `DESeqDataSet(se, design = design, ignoreRank)`: some variables in design formula are characters, converting to factors

Before we go any further we should remove any genes that have very low read counts. Here we will follow general practice of keeping rows that have at least 10 reads total. This will speed up our future calculations and avoid diluting our statistical power by doing needless tests on genes that we can not say much about anyway.

```
# chunk 4: remove genes with very low counts
# these would have never come up as statistically significant
# BUT they impact the number of tests conducted hence the adj. pval
keep.inds <- rowSums(counts(dds)) >= 10
dds <- dds[keep.inds,]
```

Now let's run the DESeq analysis pipeline on the dataset, and reassign the resulting object back to the same `dds` object:

```
# chunk 5: All-in-one - running DESeq the main function!
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

```
-- replacing outliers and refitting for 1705 genes
-- DESeq argument 'minReplicatesForReplace' = 7
-- original counts are preserved in counts(dds)
```

estimating dispersions

fitting model and testing

```
# equivalent to below:
#dds <- estimateSizeFactors(dds)
#dds <- estimateDispersions(dds)
#dds <- nbinomWaldTest(dds)
```

Here, we're running the DESeq pipeline on the `dds` object and it can take some time. The `DESeq()` function calls a number of other functions within the package to essentially run the entire pipeline (normalizing by library size by estimating the “size factors,” estimating dispersion for the negative binomial model, and fitting models and getting statistics for each gene for the design specified when we imported the data).

DESeq2 offers two kinds of hypothesis tests: the Wald test, where we use the estimated standard error of a log2 fold change to test if it is equal to zero, and the likelihood ratio test (LRT). The LRT examines two models for the counts, a *full* model with a certain number of terms and a *reduced* model, in which some of the terms of the *full* model are removed. The test determines if the increased likelihood of the data using the extra terms in the *full* model is more than expected if those extra terms are truly zero.

Since we've got a fairly simple design (single factor, two groups, lupus vs control), we can get results out of the object simply by calling the `results()` function on the `DESeqDataSet` that has been run through the pipeline.

The help page for `?results` and the vignette both have extensive documentation about how to pull out the results for more complicated models (multi-factor experiments, specific contrasts, interaction terms, time courses, etc.).

```
# chunk 6: take a peek at all the results - top of the file only
res <- results(dds)
head(res)
```

log2 fold change (MLE): state lupus vs control

Wald test p-value: state lupus vs control

DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj
--	----------	----------------	-------	------	--------	------

	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
5S_rRNA	8.668615	0.0951538	0.170068	0.559505	0.5758170	0.935789
7SK	0.733043	-0.5730441	0.617795	-0.927564	0.3536339	NA
A1BG	91.745817	-0.2840853	0.150951	-1.881973	0.0598397	0.570403
A1BG-AS1	25.942398	0.3101825	0.230798	1.343956	0.1789626	0.758740
A1CF	1.017252	-0.3106790	0.410694	-0.756474	0.4493650	NA
A2M	8.605330	-0.1413624	0.629530	-0.224552	0.8223276	0.979400

We can summarize some basic tallies using the `summary()` function. And with a different `alpha` (adjusted p-value) threshold of 0.05:

```
# chunk 7: summarize before and after filtering for significant DE genes
summary(res)
```

```
out of 32472 with nonzero total read count
adjusted p-value < 0.1
LFC > 0 (up)      : 173, 0.53%
LFC < 0 (down)    : 37, 0.11%
outliers [1]      : 0, 0%
low counts [2]    : 13381, 41%
(mean count < 4)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

```
res_p05 <- results(dds, alpha=0.05)
summary(res_p05)
```

```
out of 32472 with nonzero total read count
adjusted p-value < 0.05
LFC > 0 (up)      : 121, 0.37%
LFC < 0 (down)    : 14, 0.043%
outliers [1]      : 0, 0%
low counts [2]    : 16528, 51%
(mean count < 10)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

Let's finally extract our top genes of interest for the **THIS CLASS!**

```
# chunk 8: filter genes more and create DESeq output files
# only extracting "upregulated genes" in SLE and at least 2 fold change
top.genes <- as.data.frame(res) %>% filter(padj <= 0.05 & log2FoldChange >= 1)
dim(top.genes)
```

```
[1] 54 6
```

```
write.csv(top.genes, file=file.path(path_output, "deseq_results_topgenes.csv"))

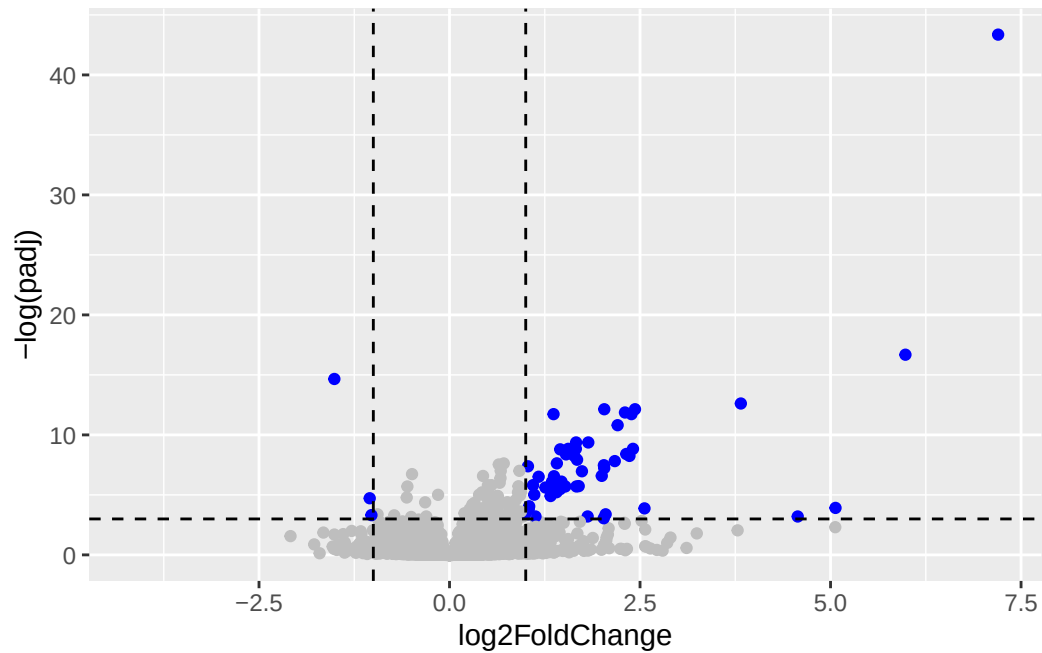
#save(top.genes, file = "top_genes.RData")

# Comprehensive file with all genes and DE statistics
# TO BE USED in the next exercise
write.csv(res, file=file.path(path_output, "deseq_results.csv"))

results <- as.data.frame(res)
mycols <- rep("gray", nrow(res))
mycols[ abs(res$log2FoldChange) > 1] = "blue"
mycols[ res$padj > 0.05 ] = "gray"

ggplot(results) +
  aes(log2FoldChange, -log(padj)) +
  geom_point(col=mycols) +
  geom_vline(xintercept = c(-1,+1), linetype=2) +
  geom_hline(yintercept = -log(0.05), linetype=2)
```

Warning: Removed 13381 rows containing missing values (`geom\_point()`).



OPTIONAL: compare IFN positive vs IFN negative samples

```
table(colData$ifn_status)
```

```
HC IFNneg IFNpos
24   34   30
```

```
# Select only IFNneg and IFNpos count columns in colData
ifn_colData <- filter(colData, ifn_status %in% c("IFNneg", "IFNpos"))
#ifn_colData <- filter(colData, ifn_status %in% c("HC", "IFNpos"))
table(ifn_colData$ifn_status)
```

```
IFNneg IFNpos
34     30
```

```
# Select only IFNneg and IFNpos count columns in counts data
ifn_countData <- dplyr::select(counts.subset, rownames(ifn_colData))
dim(ifn_countData)
```

```
[1] 56269    64
```

```
dim(ifn_colData)
```

```
[1] 64  4
```

```
# create DESeq dds object and run DESeq
ifn_dds <- DESeqDataSetFromMatrix(countData = ifn_countData,
                                   colData = ifn_colData,
                                   design = ~ifn_status)
```

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in design formula are characters, converting to factors

```
ifn_dds <- DESeq(ifn_dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

-- replacing outliers and refitting for 1817 genes

-- DESeq argument 'minReplicatesForReplace' = 7

-- original counts are preserved in counts(dds)

estimating dispersions

fitting model and testing

```
ifn_res <- results(ifn_dds)
```

```
summary(ifn_res)
```

```
out of 37778 with nonzero total read count
adjusted p-value < 0.1
LFC > 0 (up)      : 1038, 2.7%
LFC < 0 (down)    : 838, 2.2%
outliers [1]      : 0, 0%
low counts [2]    : 15950, 42%
(mean count < 2)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

```
ifn_res_p05 <- results(ifn_dds, alpha=0.05)
summary(ifn_res_p05)
```

```
out of 37778 with nonzero total read count
adjusted p-value < 0.05
LFC > 0 (up)      : 837, 2.2%
LFC < 0 (down)    : 615, 1.6%
outliers [1]      : 0, 0%
low counts [2]    : 15950, 42%
(mean count < 2)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

```
top.genes <- as.data.frame(ifn_res) %>% filter(padj <= 0.05 & log2FoldChange >= 1)
dim(top.genes)
```

```
[1] 215    6
```

```
write.csv(top.genes, file=file.path(path_output, "ifn_deseq_results_topgenes.csv"))

write.csv(ifn_res, file=file.path(path_output, "ifn_deseq_results.csv"))
```